Introduction

The following report highlights the process of designing and implementing a scalable and secure cloud architecture on the Amazon Web Services (AWS) platform for WinLocal Giveaways, a start-up based in Dublin, Ireland, offering online contests. The organization recognized the urgent requirement to move away from its aging on-premises architecture, characterized by frequent service interruptions, unmanaged database expansion, constrained scaling capabilities, and most importantly, a ransomware infection that remained unidentified for ten weeks due to the organization's lack of network segmentation

This work was done using five tasks, which involved creating an official AWS architectural diagram, developing a PoC environment in the AWS console and validating its functionality, supporting justification for two extra security solutions, considering sustainability issues within the AWS Well-Architected Framework approach, and matching the organization's security stance to the NIST Cybersecurity Framework 2.0 guidelines. All the resources were created using the us-east-1 (N. Virginia) region and named according to my identifier B00169348 for traceability and governance

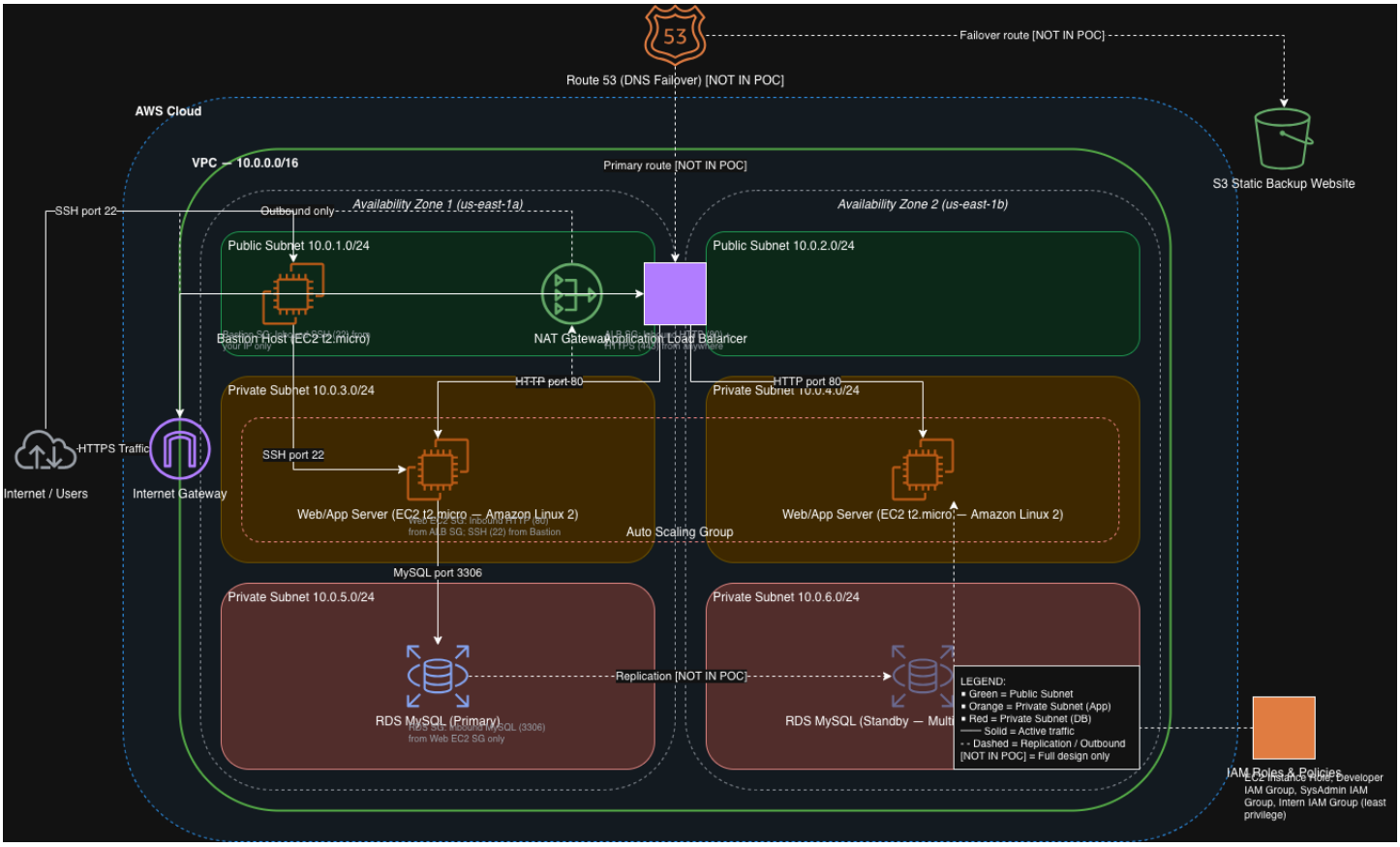
The migration process will replace the existing architecture of the NetScaler load balancer, Windows Server 2016 with IIS, and Microsoft SQL Server database with a contemporary, cloud-native 3-tier architecture that consists of Application Load Balancer, Linux EC2 instance within Auto Scaling Group, and Amazon RDS MySQL. The VPC (vpc-08c2717f57ad6f967) is the base for the whole infrastructure. All resources were developed under the cost limit of $50 USD.

Aws architecture diagram

The architectural diagram was created using draw.io with the AWS 2026 icon pack and depicts the entire target-state infrastructure for WinLocal Giveaways. The infrastructure is based on a three-tier architecture design implemented across two Availability Zones, namely us-east-1a and us-east-1b, within one virtual private cloud (VPC) using the CIDR notation 10.0.0.0/16.

The green layer is where the Application Load Balancer, Bastion Host EC2, and NAT Gateway reside. The orange layer is home to the web server Auto Scaling Group, which is isolated from the Internet. Lastly, the red/pink layer comprises the primary RDS MySQL instance in us-east-1a and the replica in us-east-1b, identified as not within the PoC scope.

The external elements that are found outside the scope of VPC include Route 53 for the DNS failover (indicated as outside PoC), an S3 bucket for the static website failover, IAM roles and policies, as well as the users/internet icon. The security group markings have been included directly next to each element with information about what ports and from where traffic is allowed, emphasizing the principle of least privilege at all layers. Arrows depicting the direction of traffic have been used, solid for actual traffic and dotted for one-way/replication traffic.



Proof of concept implementation

Proof of Concept shows that the necessary components of the AWS architecture needed for solving all four issues of WinLocal have been created. All components have been created in us-east-1 region within VPC.

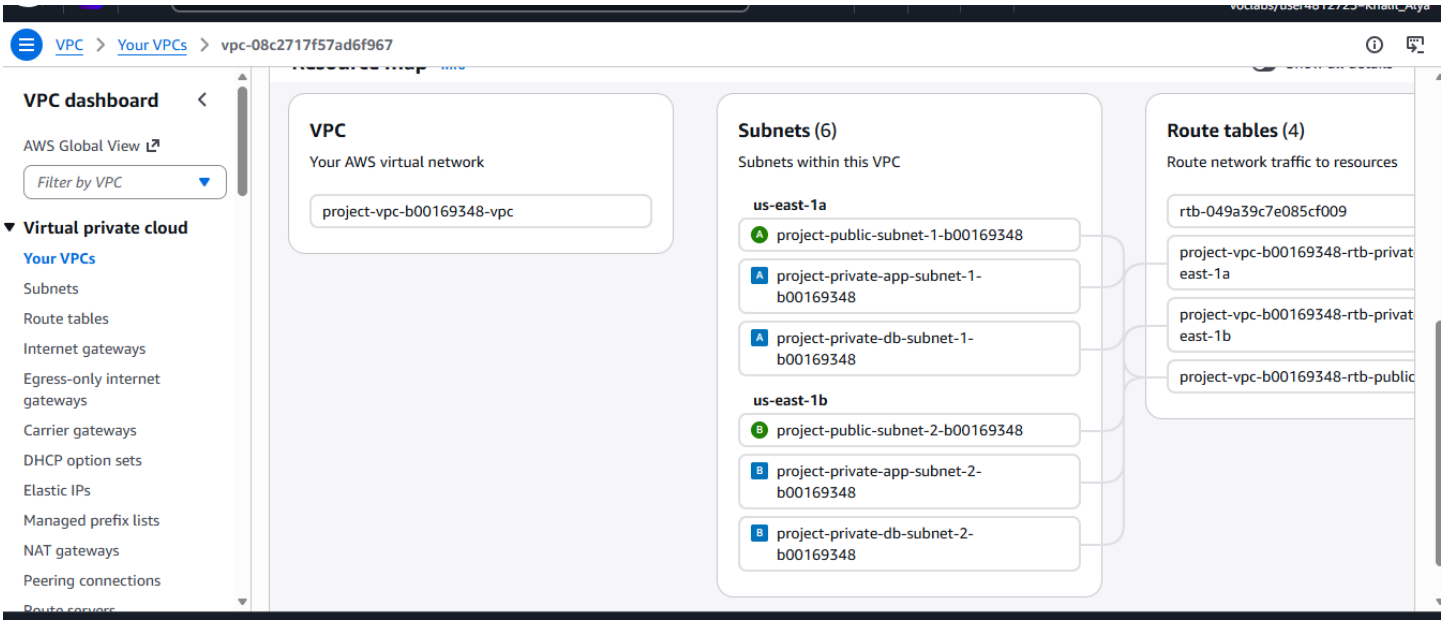
Networking

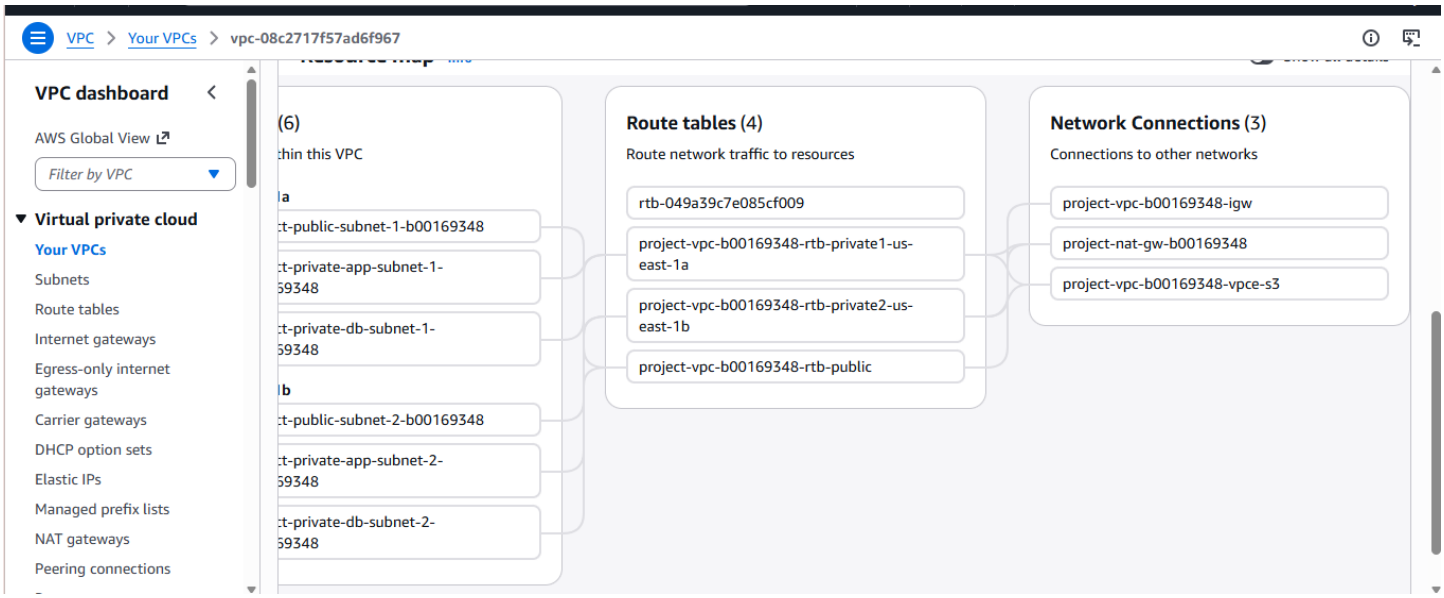
The networking layer provides the basic structure of the entire design process. The creation of a customized VPC (project-vpc-b00169348-vpc, ID: vpc-08c2717f57ad6f967) is achieved using the CIDR 10.0.0.0/16, which provides 65,536 IP addresses. DNS hostnames and DNS Resolution are activated in order to facilitate name-based

In the first round of subnet creation, four subnets belonging to an earlier session with improper CIDR blocks (/20) were discovered in the VPC and deleted. After that, six subnets were properly established in two Availability Zones by dividing the subnet into /24 – two public (10.0.1.0/24 and 10.0.2.0/24), two application (10.0.3.0/24 and 10.0.4.0/24), and two database subnets (10.0.5.0/24 and 10.0.6.0/24) in us-east-1a and us-east

An Internet Gateway (igw-013ed6e326da383f2) was added to the VPC, which will help route internet-bound traffic both ways for the public resources. A NAT Gateway (nat-0b04ce4c9b586fd36, Elastic IP: 52.4.160.60) was set up

within the first public subnet via Zonal connectivity, through which EC2 instances in the private subnets can access the internet without exposing themselves to the internet world. There are three route tables set up: the public route table will route traffic at 0.0.0.0/0 to the Internet Gateway, and it is linked to both public subnets; similarly, the two private route tables will route traffic at 0.0.0.0/0 to the NAT Gateway.

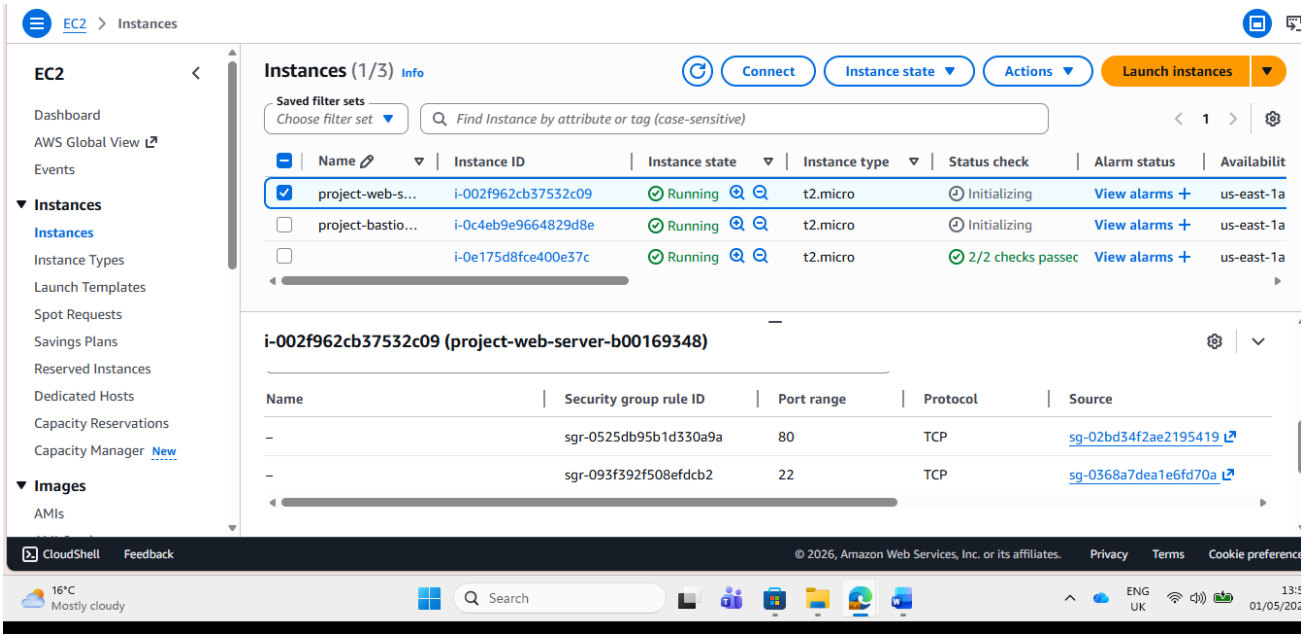


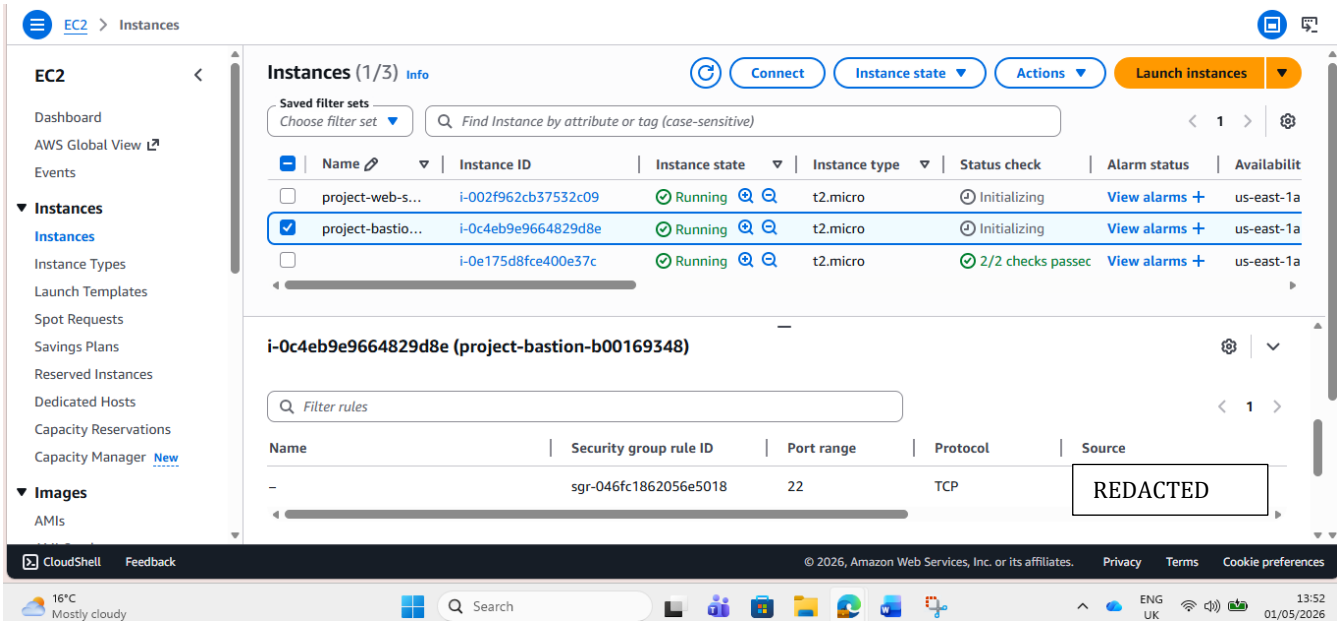


Bastion host

The Bastion host (project-bastion-b00169348) was created using an Amazon Linux 2 t2.micro Amazon EC2 instance in the first public subnet with a public IP address assigned to it. The security group used for this Bastion host (project-bastion-sg-b00169348) has SSH access to port 22 blocked to all but one trusted IP address, which is that of the administrator's computer, making sure that there is no other way to enter the network except through this Bastion host.

A second EC2 instance called project-web-server-b00169348 was deployed in the application subnet (10.0.3.0/24), which does not have a public IP address, meaning that it cannot be accessed through the Internet at all. In order to manage such an instance, the first SSH connection needs to be established to the Bastion Host, followed by another one that will connect to the private IP address of the server in the private subnet. This jump-host approach is precisely how WinLocal's Issue 4 could be mitigated, a ransomware attack has been performed by exploiting the flat network design.

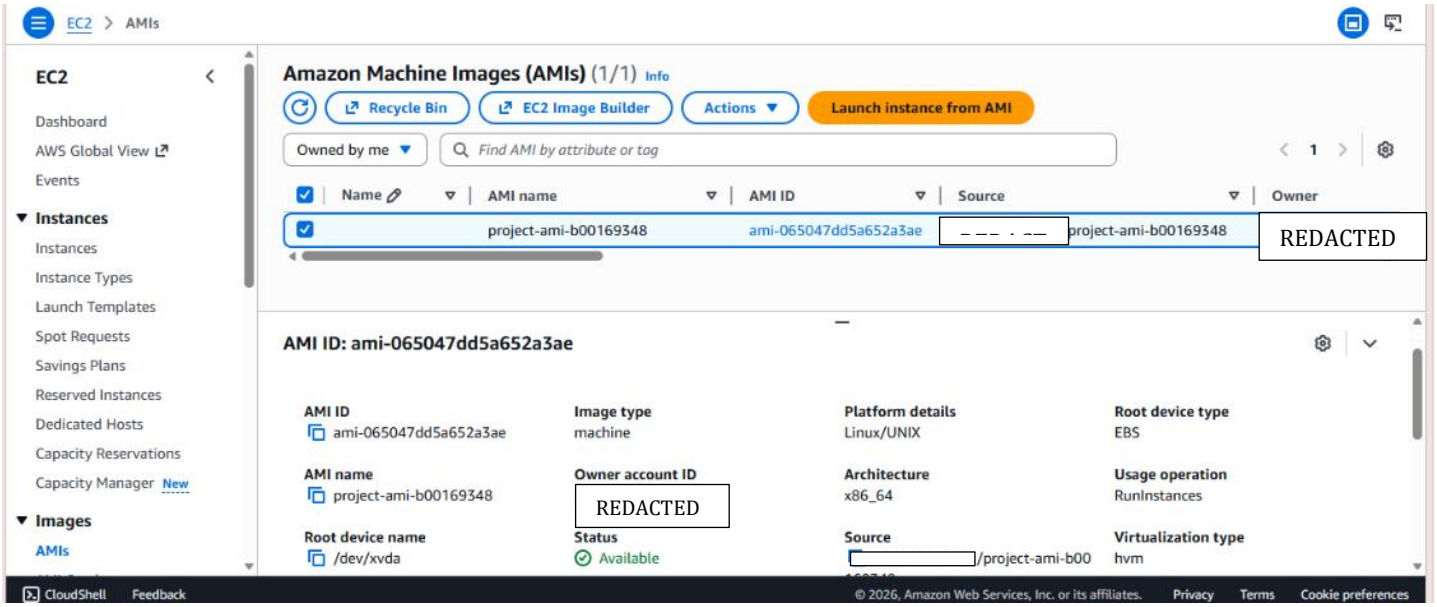




Ami creation

The creation of an Amazon Machine Image was done through the web server EC2 instance configured through the selection of the instance followed by clicking on Actions > Image and Templates > Create Image. An Amazon Machine Image captures all the details regarding a particular running instance that includes the entire state of the instance including the OS, software and application code.

The AMI acts as the source image used by the Launch Template for the Auto Scaling Group. When the ASG has the need to create an additional instance to help cope with higher load, it creates an exact copy of the original web server based on the AMI that is fully configured and does not require any kind of setup whatsoever since it comes up running and able to process requests right away. There is no configuration drift across the fleet of instances and there are no differences between individual instances; immutable infrastructure policies can be implemented easily using this method.

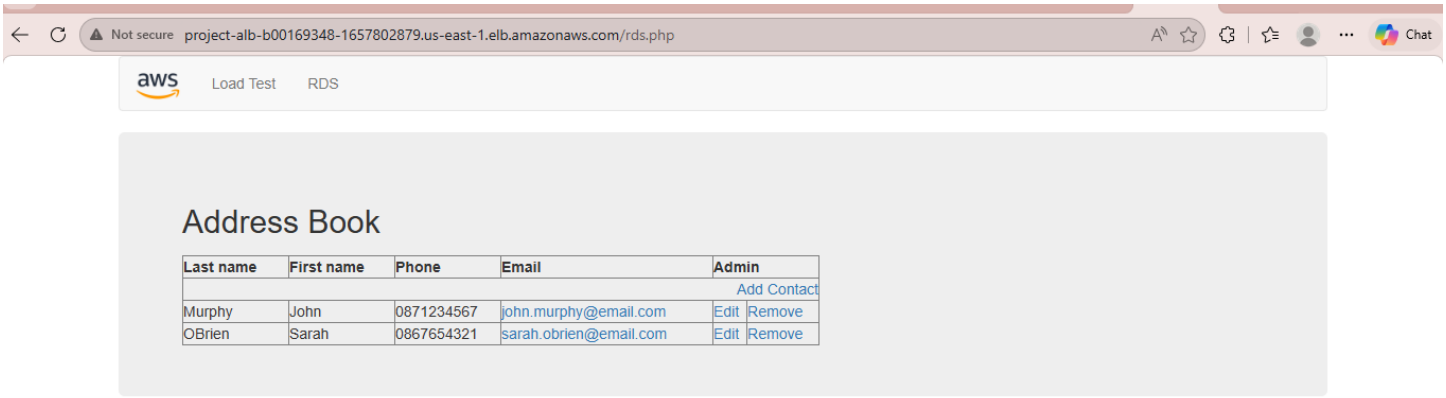


Rds my sql

The Amazon RDS service was employed to replace WinLocal's local Microsoft SQL Server, which was facing issues such as running out of disk space and having to be manually managed all the time. The DB Subnet Group (project-db-subnet-group-b00169348) is configured to use both private database subnets (10.0.5.0/24 and 10.0.6.0/24) in order to isolate the database fully from the Internet. The RDS MySQL (project-rds-b00169348) was created using db.t3.micro with 20 GiB gp2 storage with the name of the database being winlocal.

Single-AZ was chosen as the PoC option due to staying under the budget constraint of $50. Single-AZ means running an RDS instance on one AZ without any failover. When such an AZ is down, the database is down until AWS brings it back up, which is the main issue related to the PoC setup. Multi-AZ should be used in production mode:

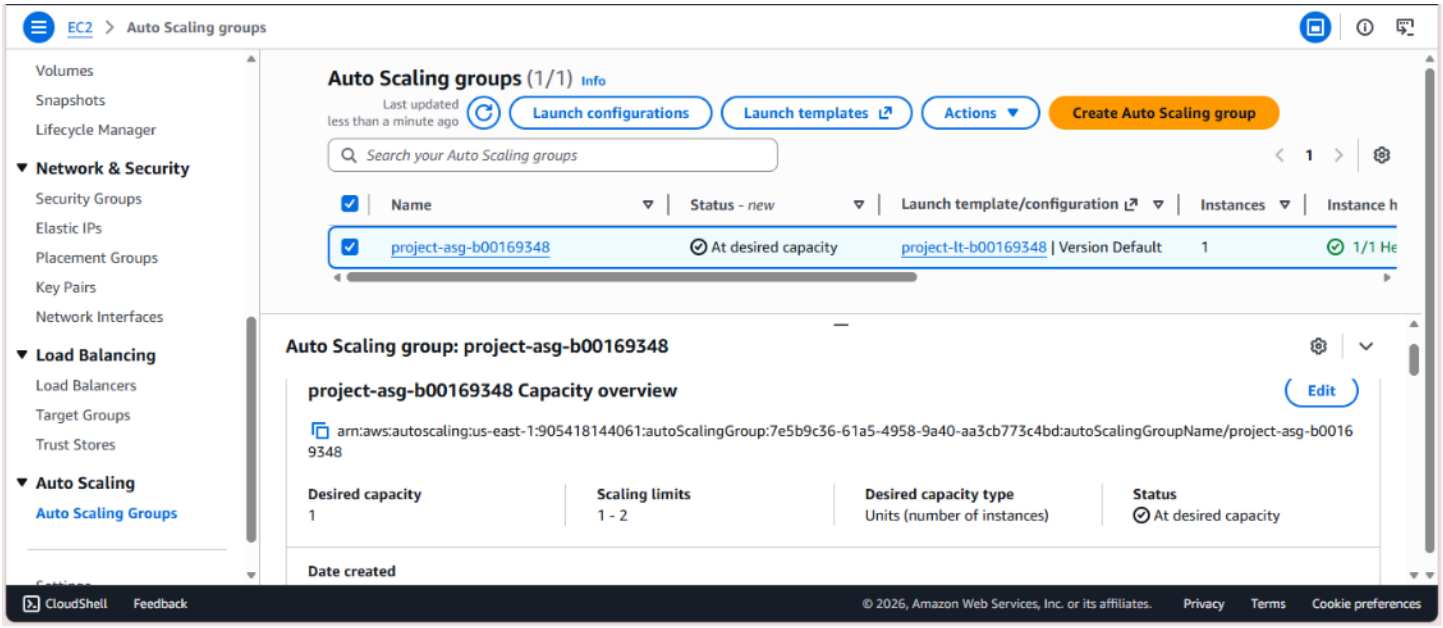
RDS creates a synchronized database on a second AZ, which becomes the primary database automatically after 60-120 seconds since the first one is down. It will solve WinLocal's Issue 2: there won't be a single point of failure anymore. Web Application was set up to use an RDS endpoint, and it has been shown to display data from winlocal database.

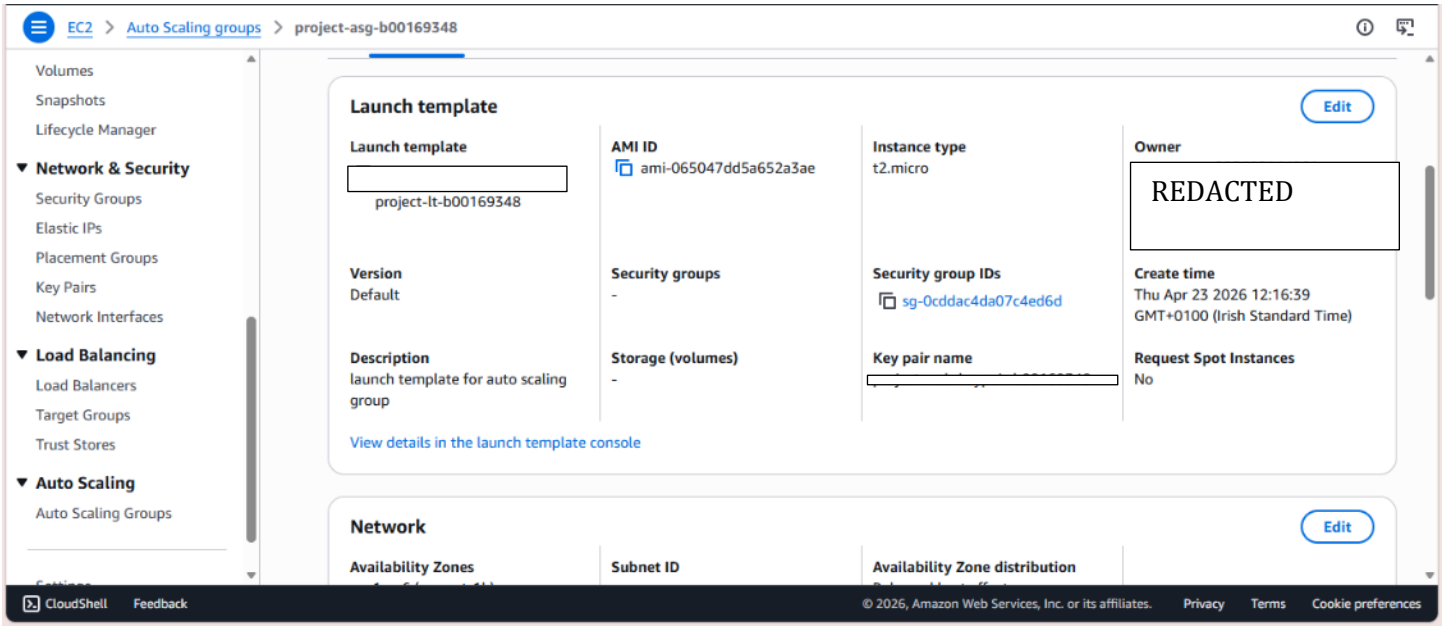


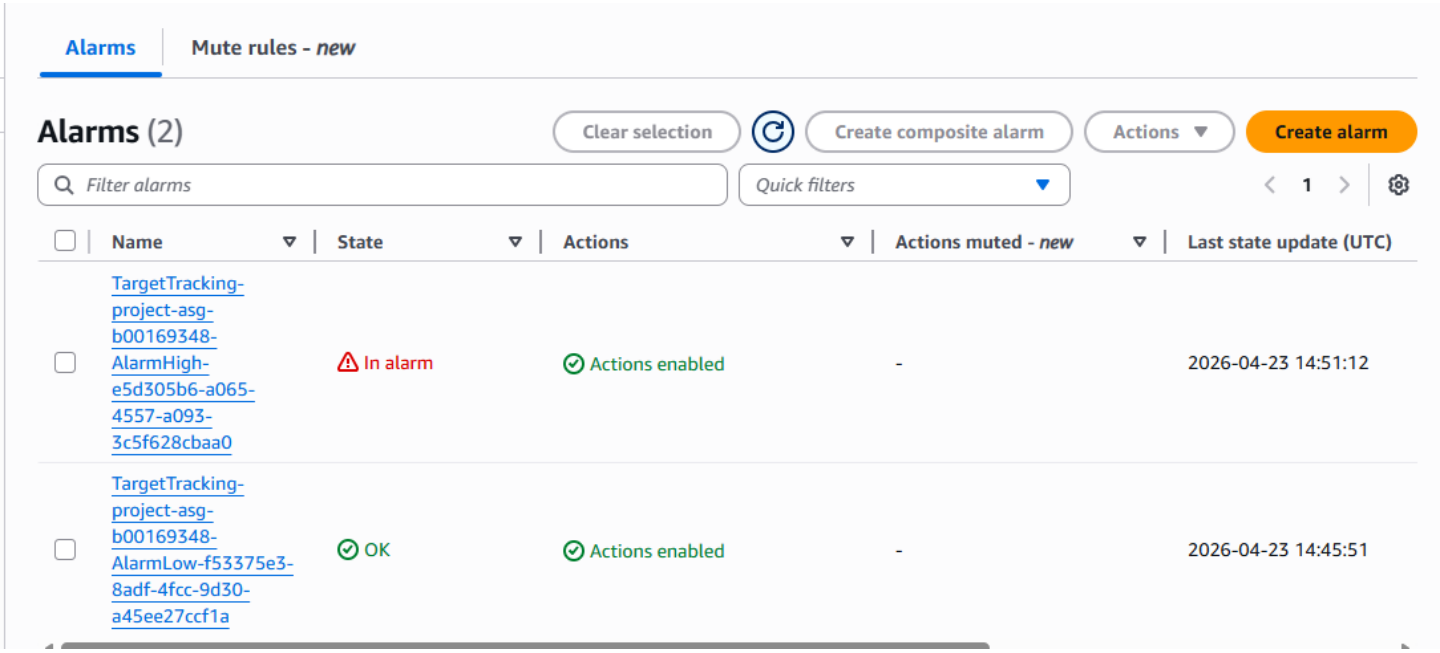
Auto scaling group

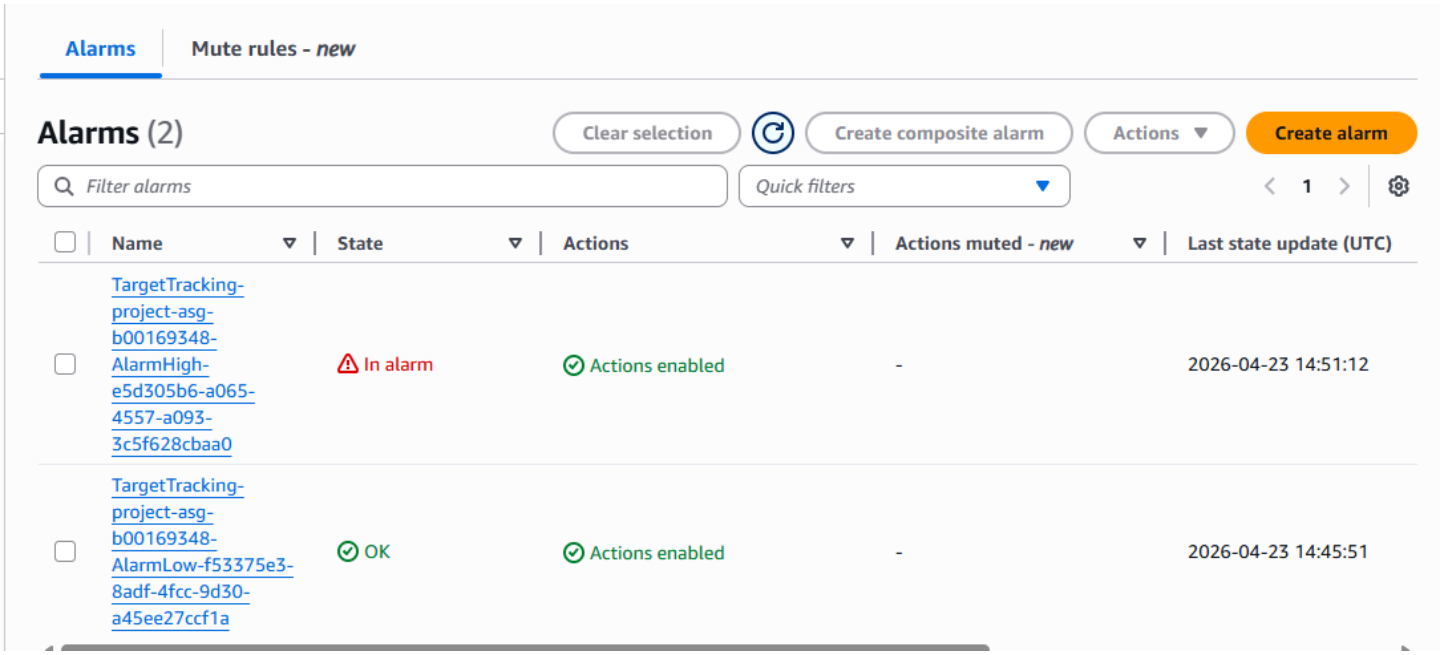
An Auto Scaling Group (project-asg-b00169348) was created using the Launch Template (project-lt-b00169348) created using the custom AMI. Desired capacity was specified as 1, minimum as 1, and maximum as 2. Target Tracking scaling policy was used for an average target CPU usage of 60%. If the CPU utilization goes beyond 60%, then the extra load is detected by the CloudWatch system, and accordingly, a new instance of EC2 based on the AMI is launched within a private application subnet, which joins the target group of the ALB. In case the load goes down below the specified value, the new instance is removed.

For proving the scaling strategy, the Stress tool for Linux was installed in the instance and was run so that it created some CPU usage which was more than 60%. Within two to five minutes after that, a CloudWatch Alarm was found to be in the ALARM state and there was a record of a scale out action in the ASG activity logs with a new instance being created.


EC2 > Auto Scaling groups
Volumes
Snapshots
Lifecycle Manager
Network & Security
Security Groups
Elastic IPs
Placement Groups
Key Pairs
Network Interfaces
Load Balancing
Load Balancers
Target Groups
Trust Stores
Auto Scaling
Auto Scaling Groups
Auto Scaling groups (1/1) Info
Last updated less than a minute ago
Launch configurations
Launch templates
Actions
Create Auto Scaling group
Search your Auto Scaling groups
Name
Status - new
Launch template/configuration
Instances
project-asg-b00169348
At desired capacity
project-lt-b00169348 | Version Default
1
Auto Scaling group: project-asg-b00169348
project-asg-b00169348 Capacity overview
Edit
arn:aws:autoscaling:us-east-1:905418144061:autoScalingGroup:7e5b9c36-61a5-4958-9a40-aa3cb773c4bd:autoScalingGroupName/project-asg-b00169348
Desired capacity
1
Scaling limits
1 - 2
Desired capacity type
Units (number of instances)
Status
At desired capacity
Date created
CloudShell
Feedback
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences


EC2 > Auto Scaling groups > project-asg-b00169348
Volumes
Snapshots
Lifecycle Manager
Network & Security
Security Groups
Elastic IPs
Placement Groups
Key Pairs
Network Interfaces
Load Balancing
Load Balancers
Target Groups
Trust Stores
Auto Scaling
Auto Scaling Groups
Launch template
Edit
Launch template
project-lt-b00169348
AMI ID
ami-065047dd5a652a3ae
Instance type
t2.micro
Owner
REDACTED
Version
Default
Security groups
-
Security group IDs
sg-0cddac4da07c4ed6d
Create time
Thu Apr 23 2026 12:16:39 GMT+0100 (Irish Standard Time)
Description
launch template for auto scaling group
Storage (volumes)
-
Key pair name
Request Spot Instances
No
View details in the launch template console
Network
Edit
Availability Zones
Subnet ID
Availability Zone distribution
CloudShell
Feedback
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences



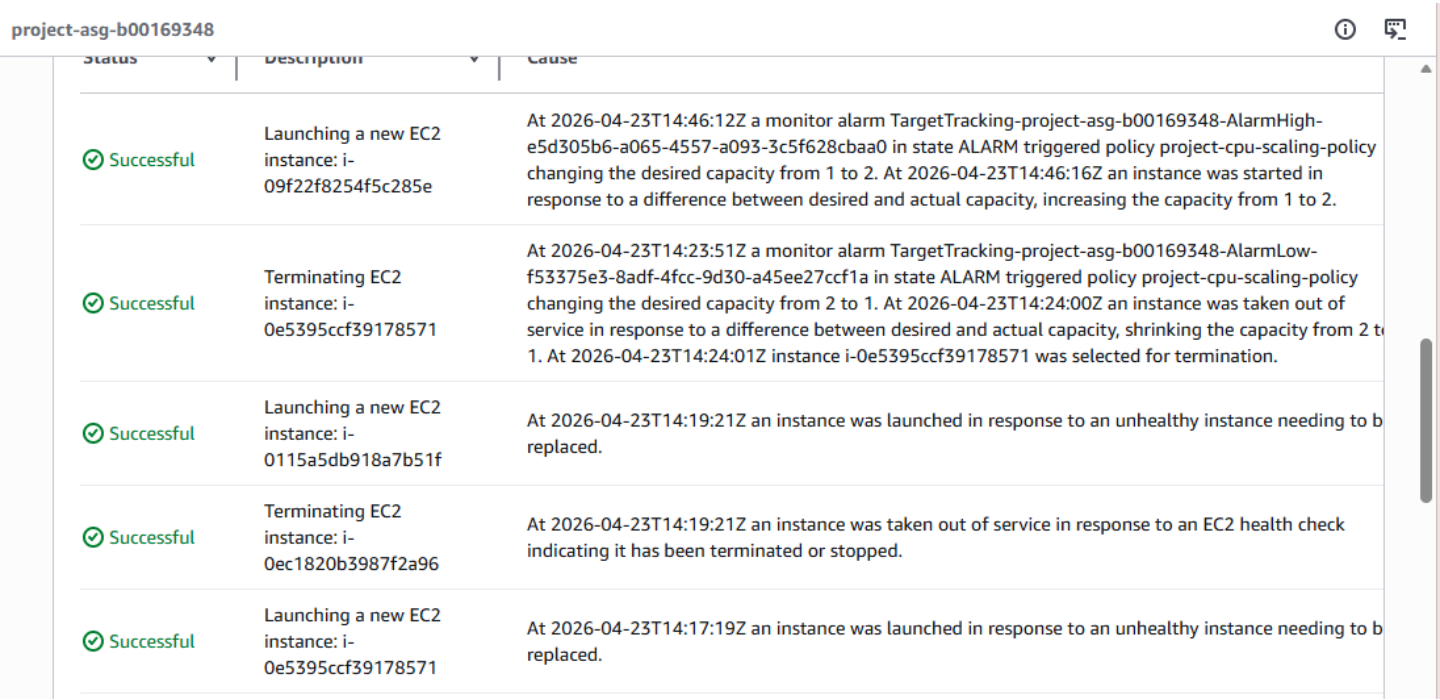


Application load balancer

To overcome Issue 1, an Application Load Balancer (project-alb-b00169348) was deployed to both public subnets, replacing the aged and inefficient NetScaler Load Balancer utilized by WinLocal. This ALB allows incoming HTTP connections over port 80 and HTTPS connections over port 443 from the internet, forwarding these connections to the Target Group (project-tg-b00169348) that is directly associated with the Auto Scaling Group. The Target Group undergoes a health check, and the ALB will automatically stop forwarding connections to unhealthy instances. As

new instances are spawned, they will be added to the target group, allowing for zero downtime scaling without a single point of failure at the application level.

In order to validate that the load balancing is done, the URL for the ALB was accessed through a web browser and kept refreshing. Each page shows the Instance ID of the EC2 instance that served that request. Different instance IDs were seen each time the page was refreshed.



Security implementation

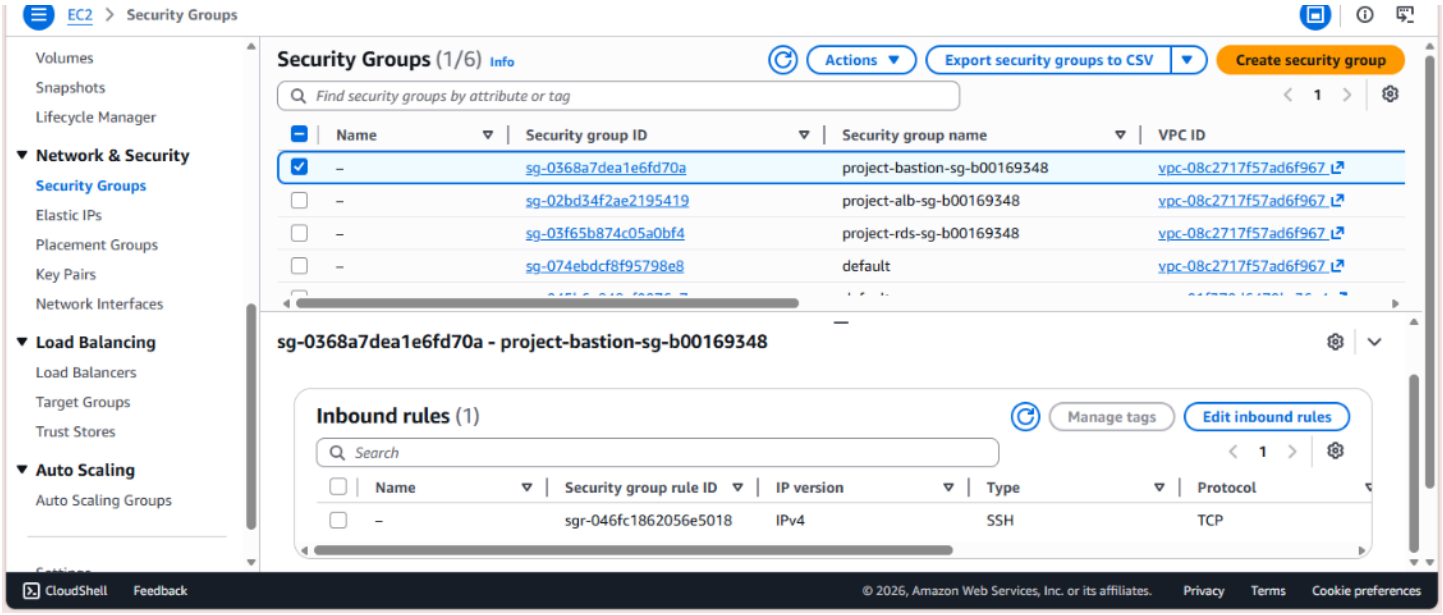
Layered security approach was used, whereby four AWS Security Groups were created, one per each layer in the architecture. AWS Security Groups are stateful virtual firewalls assigned to AWS resources, and rule evaluation is done individually for each resource. This provides finer-grained access control than just having ACL at the subnet level. The principle of least privilege has been followed; resources can accept network traffic only from those sources that are absolutely necessary.

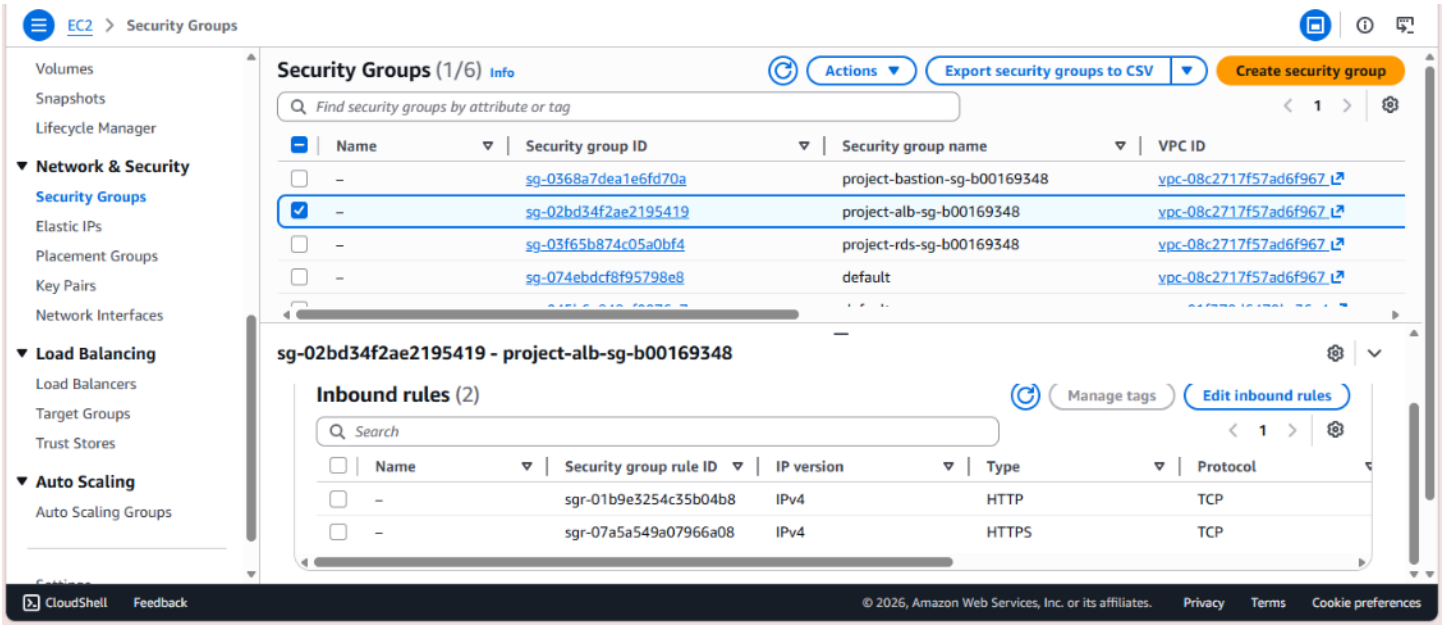
Security group	Attached to	Inbound rule	Source
project-alb-sg-b00169348	Application Load Balancer	HTTP 80, HTTPS 443	0.0.0.0/0 (Internet)
project-bastion-sg-b00169348	Bastion Host EC2	SSH 22	Administrator IP only
project-web-ec2-sg-b00169348	Web Server EC2 (ASG)	HTTP 80	project-alb-sg-b00169348
project-web-ec2-sg-b00169348	Web Server EC2 (ASG)	SSH 22	project-bastion-sg-b00169348

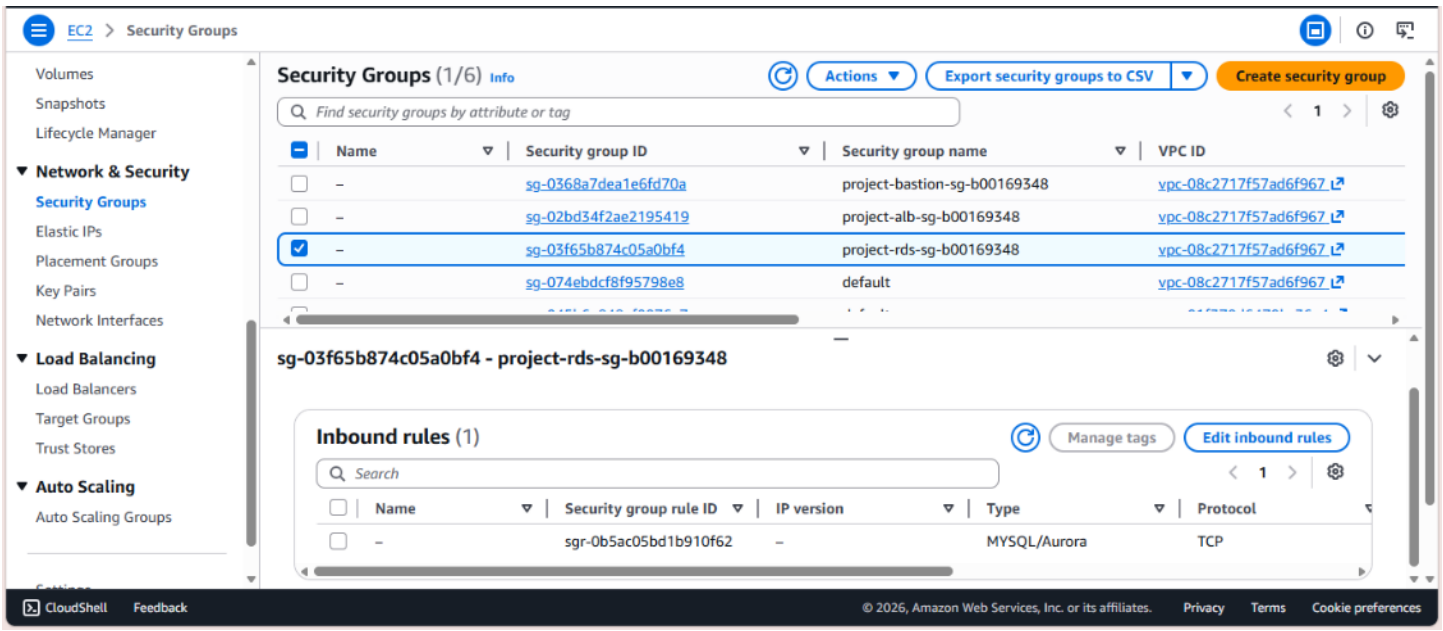
project-rds-sg-b00169348	RDS MySQL	MySQL 3306	project-web-ec2-sg-b00169348

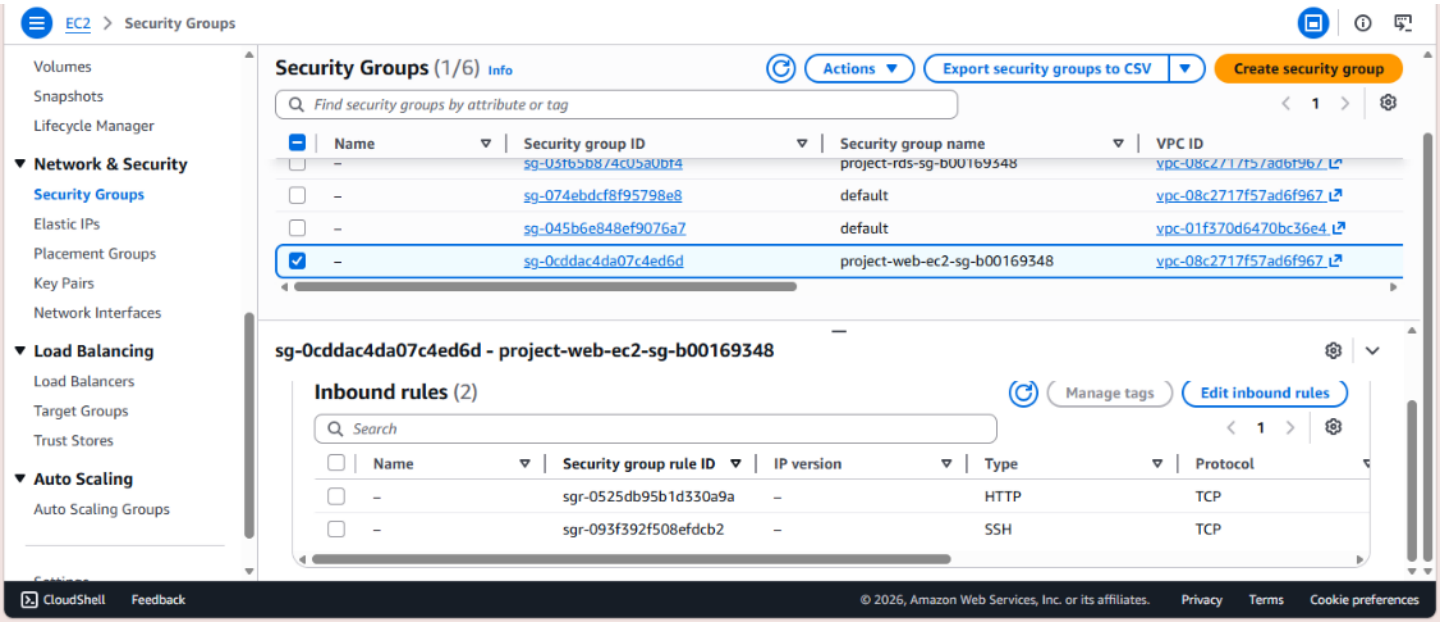
Using security groups to be references to rule sources instead of IP address ranges through CIDR notation is one of the important design considerations. This means that when there will be changes such as replacing or scaling up the EC2 instances using the ASG, the rules will stay effective without any need for changes. This multilayered design will ensure that even if the attacker manages to access the ALB, it won't provide access to EC2 instances on random ports. In addition, even if an attacker compromises an EC2 instance, it won't be able to access any other resource other than the database listening on port 3306.

Public subnets have ALB and Bastion only.




EC2 > Security Groups
Volumes
Snapshots
Lifecycle Manager
Network & Security
Security Groups
Elastic IPs
Placement Groups
Key Pairs
Network Interfaces
Load Balancing
Load Balancers
Target Groups
Trust Stores
Auto Scaling
Auto Scaling Groups
Security Groups (1/6) Info
Actions
Export security groups to CSV
Create security group
Find security groups by attribute or tag
Name
Security group ID
Security group name
VPC ID
sg-0368a7dea1e6fd70a
project-bastion-sg-b00169348
vpc-08c2717f57ad6f967
sg-02bd34f2ae2195419
project-alb-sg-b00169348
sg-03f65b874c05a0bf4
project-rds-sg-b00169348
sg-074ebdcf8f95798e8
default
sg-02bd34f2ae2195419 - project-alb-sg-b00169348
Inbound rules (2)
Manage tags
Edit inbound rules
Search
Security group rule ID
IP version
Type
Protocol
sgr-01b9e3254c35b04b8
IPv4
HTTP
TCP
sgr-07a5a549a07966a08
IPv4
HTTPS
TCP
CloudShell
Feedback
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences

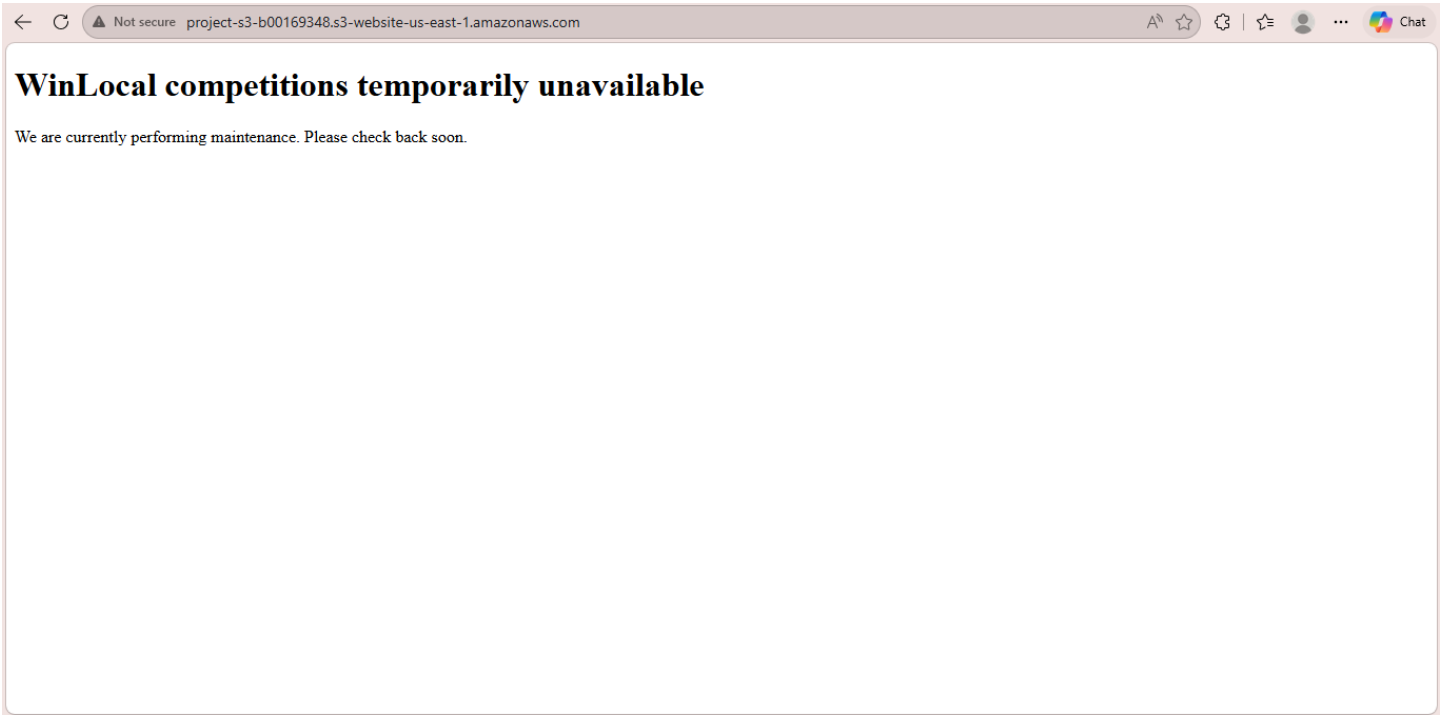

EC2 > Security Groups
Volumes
Snapshots
Lifecycle Manager
Network & Security
Security Groups
Elastic IPs
Placement Groups
Key Pairs
Network Interfaces
Load Balancing
Load Balancers
Target Groups
Trust Stores
Auto Scaling
Auto Scaling Groups
Security Groups (1/6) Info
Actions
Export security groups to CSV
Create security group
Find security groups by attribute or tag
Name
Security group ID
Security group name
VPC ID
sg-0368a7dea1e6fd70a
project-bastion-sg-b00169348
vpc-08c2717f57ad6f967
sg-02bd34f2ae2195419
project-alb-sg-b00169348
sg-03f65b874c05a0bf4
project-rds-sg-b00169348
sg-074ebdcf8f95798e8
default
sg-03f65b874c05a0bf4 - project-rds-sg-b00169348
Inbound rules (1)
Manage tags
Edit inbound rules
Search
Security group rule ID
IP version
Type
Protocol
sgr-0b5ac05bd1b910f62
MYSQL/Aurora
TCP
CloudShell
Feedback
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences



S3 static backup website

Amazon S3 bucket with static website hosting set up (project-static-site-b00169348) is used as a failover target in case of failure of all the tiers of the application. Static Website Hosting on AWS S3 stores the HTML files directly on the storage service without any requirement of EC2 instance or load balancer and without having to code an application; hence, this is very high availability and low-cost setup for a maintenance page.

A public HTML file was uploaded with a message stating that competitions are not available at the moment for the WinLocal app. A public read bucket policy was implemented, and the website was accessed using the endpoint URL of the S3 storage system. For the complete architecture design, Route 53 should have a failover routing policy, where the health check will detect whether the ALB is healthy or unhealthy. If it detects that the ALB is unhealthy, Route 53 will update DNS and route the traffic to the S3 static website after 30 to 60 seconds. This issue can easily be fixed by implementing a solution like this.



Additional services

There are two services which have been depicted in the Task 1 Architecture Diagram, but they do not feature in the Proof-of-Concept due to reasons which are stated below. Both of them have been associated with a particular issue with the current infrastructure.

Aws web application layer

The AWS WAF is a managed service that acts like a firewall for web applications by checking each and every HTTP/HTTPS request sent to the Application Load Balancer before routing them to the EC2 instances. Each request is evaluated using a set of rules based on the OWASP Top 10 threats, such as SQL injection, cross-site scripting, malicious bots, and geographically restrictive content.

Managed Rulesets are provided by AWS and kept up-to-date by AWS, eliminating the need for security engineers to manage the rulesets.

With WinLocal, AWS WAF is solving Issue 4, which involves the ransomware attack. It is highly likely that this attack was executed through the exploitation of a vulnerable web asset, and such a malicious request will have been intercepted by AWS WAF long before it made it to the application level. Also, AWS WAF provides detailed logs for requests and can deliver them to S3 or Cloudwatch services. This solution creates an audit trail that was not available at the time of the attack. It only requires the creation of a WAF Web ACL in the console and association with project-alb-b00169348.

Amazon route 53 dns failover

Route 53 is Amazon's Domain Name System (DNS) service. Using the failover routing policy, the primary DNS record points to the application load balancer while the secondary one points to the S3 static website. The Route 53 health check regularly tests the availability of the ALB endpoint. In case of failure, Route 53 automatically redirects traffic to the static site hosted on S3 in less than a minute. The recovery process for the ALB does not require any action from users.

Route 53 will fix Problem 1 in WinLocal. With the old NetScaler load balancer, any kind of failure will result in a total outage where users get connection error messages without anything else. Since there is now the Route 53 failover and the S3 static site has been created within the PoC, then when there is any kind of infrastructure failure, users will be shown the maintenance page. This will require the creation of a Route 53 hosted zone for WinLocal and the configuration of two alias records, one pointing to project-alb-b00169348 and another to the S3 endpoint.

Sustainability

"Sustainability" is one of the six pillars of the AWS Well-Architected Framework, and it deals with minimizing the environmental effect of cloud workloads by optimizing resource usage, using managed services, and avoiding idling of infrastructure. "WinLocal PoC" has adopted various sustainability practices by design with additional recommendations for production environments.

Practices implemented in the poc

The Auto Scaling feature is the most sustainable solution in the Proof of Concept. With the configuration of the minimum capacity as 1 and the maximum as 2, the ASG scales up automatically whenever the CPU usage goes below 60%, thereby ensuring that no EC2 instances remain idle. The WinLocal solution had a constant capacity irrespective of the demand in their previous on-premises solution, which meant energy consumption even without any competitions taking place.

Wherever possible, managed services were employed. The Amazon RDS and Application Load Balancer are managed by AWS using shared and optimized infrastructure, which is not something that can be replicated individually. If equivalent software is deployed manually, running in EC2 instances will consume the EC2 instances constantly but with much less utilization. Rightsizing was done in all scenarios by using t2.micro EC2 and db.t3.micro RDS, the smallest instance types that are suitable for PoC.

Future sustainability recommendations

Evaluation of AWS Graviton instances should be conducted for EC2 as well as RDS deployment in production. These processors are based on ARM architecture and have much better performance-per-watt than the corresponding x86 processors, thus reducing costs and decreasing carbon footprints without modifications of the code. Amazon S3 Intelligent-Tiering should be used for all buckets containing data related to the competition and application logs.

Instead of running EC2 all the time to perform these functions, AWS Lambda should be considered as an alternative. The application would run only when there is a trigger. It does not consume any resource when it is

idle. It is recommended that AWS Cost Explorer and Trusted Advisor be configured to discover underutilized resources to support the rightsizing process as the WinLocal website gains traffic.

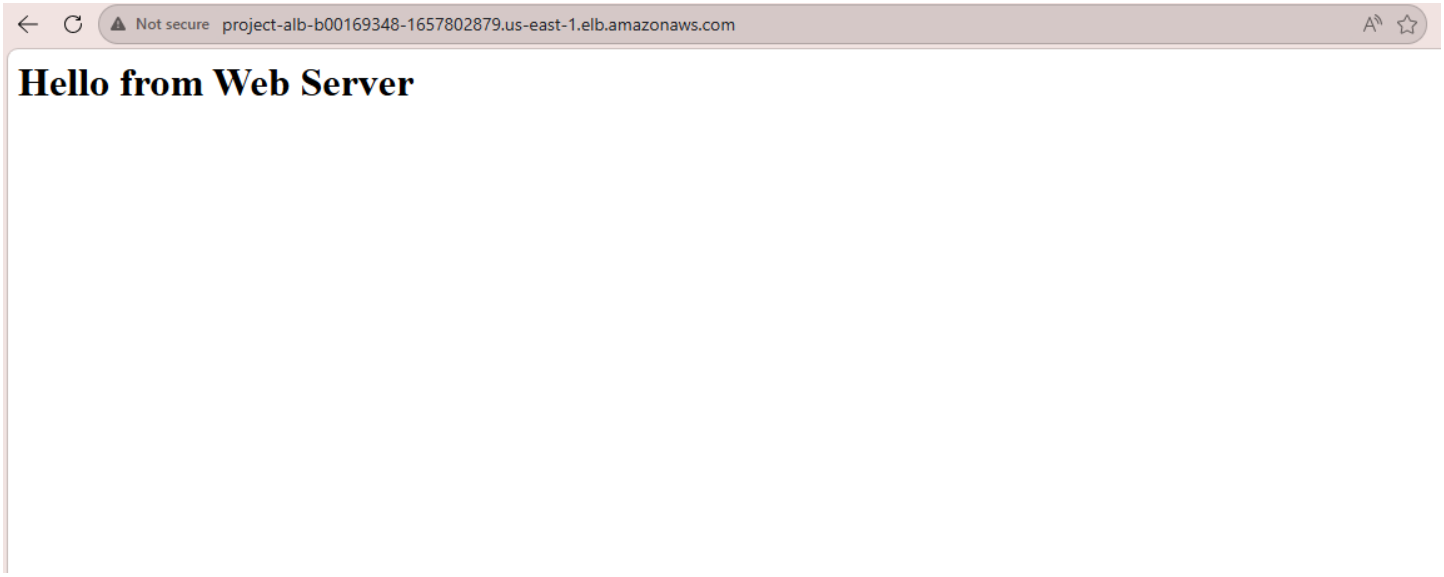
Conclusion

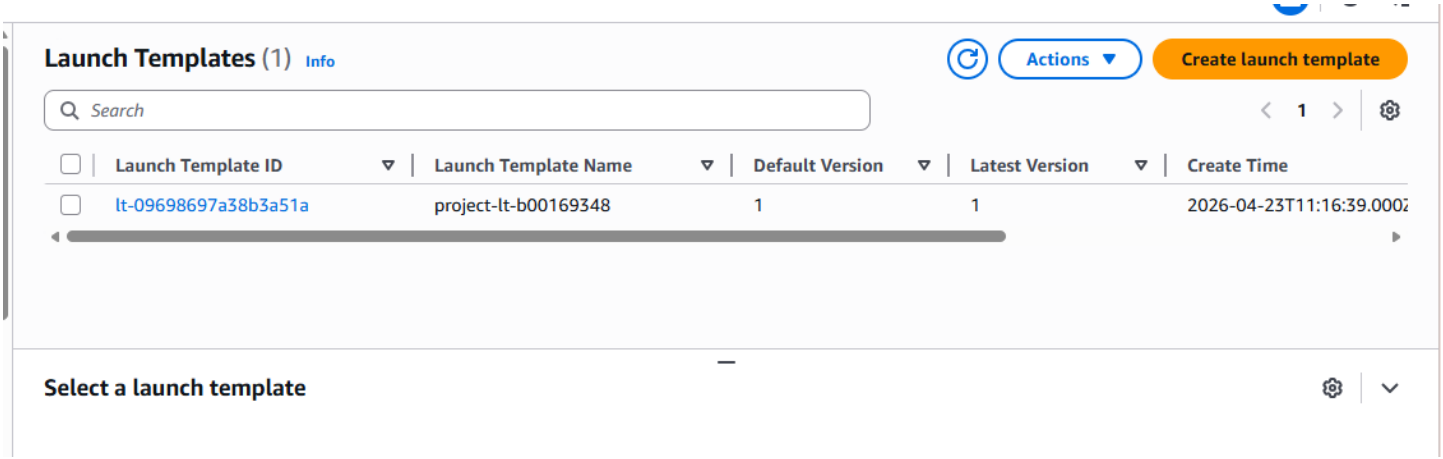
The project has successfully come up with an effective and efficient cloud infrastructure design based on Amazon Web Services (AWS) to solve all the issues associated with WinLocal Giveaways. This has been achieved by tackling all four operational problems: Firstly, through the implementation of AWS Application Load Balancer and AWS Route 53 DNS failover to tackle the problem of NetScaler; secondly, implementing AWS RDS MySQL with automatic backup feature to solve the problem of SQL server which was occupying too much disk space; thirdly, using Auto Scaling Group which solves the issue of manual scaling; and finally, implementing VPC Network Segmentation to solve the issue of flat network.

Proof of concept showed that all components work as required. The VPC (vpc-08c2717f57ad6f967) with six subnets in two Availability Zones, the Internet Gateway (igw-013ed6e326da383f2), and the NAT Gateway (nat-0b04ce4c9b586fd36) with an Elastic IP address of [REDACTED] create a correctly separated and highly available network infrastructure. The Bastion Host offers a secure connection to the private instances. The RDS MySQL instance provided successful connectivity between the sample application and the database layer. The Auto Scaling Group could scale out in case of CPU overload and scale back in. The ALB distributed incoming traffic among instances because we observed that the requests went to instances with different IDs

The inclusion of AWS WAF directly addresses the web-facing vulnerability vector that made the incident possible, while Route 53 DNS failover finishes the story on high availability. The mapping against NIST CSF 2.0 shows that while there have been considerable improvements in the Protect domain, it is the Detect and Govern domains that require the greatest investment at this time. The architecture and recommendations offered in this report provide an actionable guide for WinLocal to achieve maturity in their cybersecurity stance as they scale.

Additional screenshots

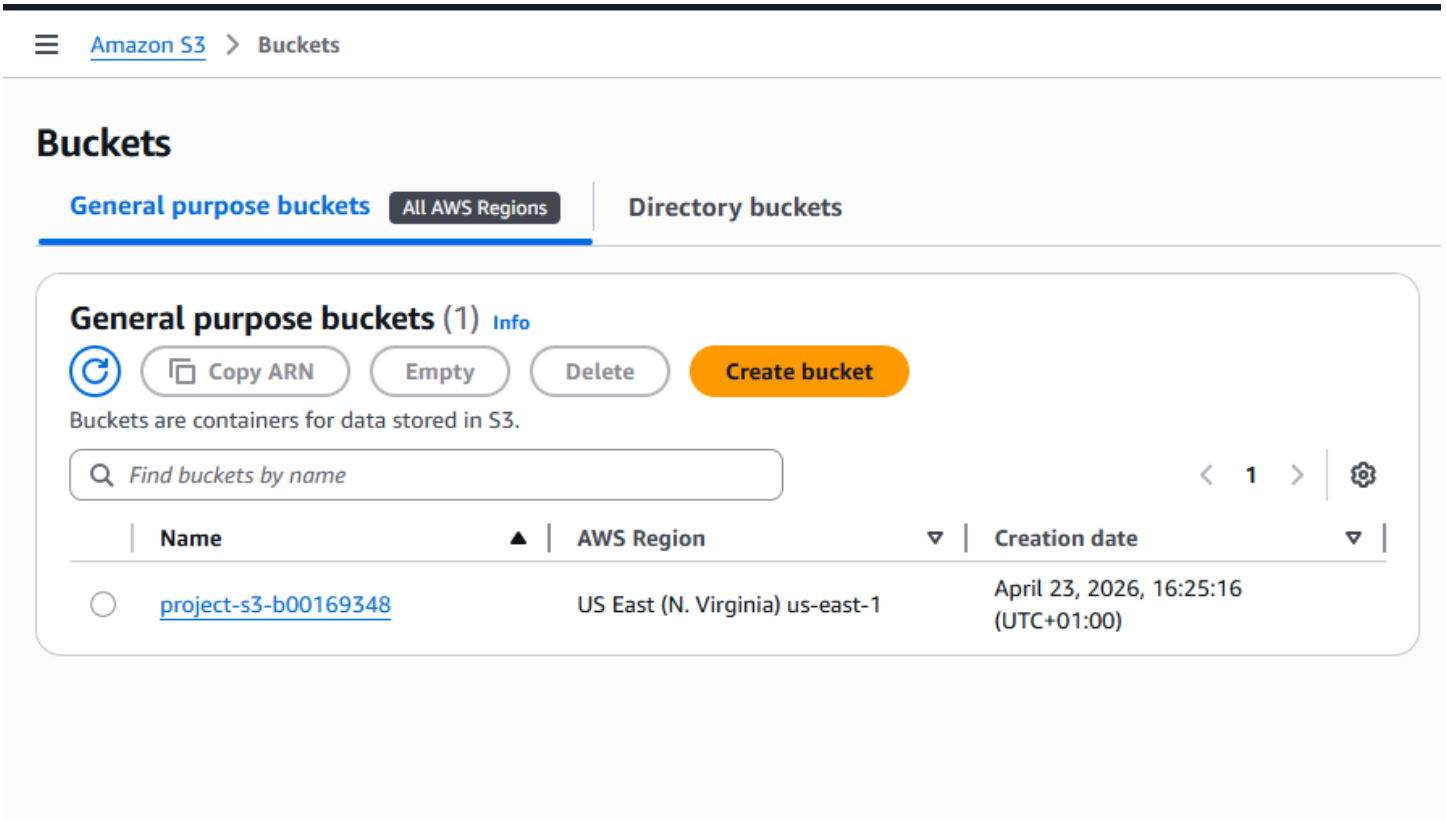

Not secure project-alb-b00169348-1657802879.us-east-1.elb.amazonaws.com
Hello from Web Server


Launch Templates (1) Info
Actions
Create launch template
Search
1
Launch Template ID
Launch Template Name
Default Version
Latest Version
Create Time
lt-09698697a38b3a51a
project-lt-b00169348
1
1
2026-04-23T11:16:39.000
Select a launch template

```
stress --cpu 2 --timeout 300
Last metadata expiration check: 0:12:18 ago on Thu Apr 23 13:08:58 2026.
Dependencies resolved.
================================================================================
 Package              Architecture          Version                    Repository                  Size
================================================================================
Installing:
 stress               x86_64                1.0.7-2.amzn2023.0.1       amazonlinux                 34 k

Transaction Summary
================================================================================
Install  1 Package

Total download size: 34 k
Installed size: 68 k
Downloading Packages:
stress-1.0.7-2.amzn2023.0.1.x86_64.rpm                  332 kB/s |  34 kB     00:00
--------------------------------------------------------------------------------
Total                                                   213 kB/s |  34 kB     00:00
Running transaction check
Transaction check succeeded.
Running transaction test
Transaction test succeeded.
Running transaction
  Preparing        :                                                        1/1
  Installing       : stress-1.0.7-2.amzn2023.0.1.x86_64                     1/1
  Running scriptlet: stress-1.0.7-2.amzn2023.0.1.x86_64                     1/1
  Verifying        : stress-1.0.7-2.amzn2023.0.1.x86_64                     1/1
```



Routes | Subnet associations | Edge associations | Route propagation | Tags

Routes (3)

Both | Edit routes

Filter routes

Destination	Target	Status	Propagated	Route Origin
pl-63a5400a	vpce-00ae2ac99493c0652	Active	No	Create Route
0.0.0.0/0	nat-0b04ce4c9b586fd36	Active	No	Create Route
10.0.0.0/16	local	Active	No	Create Route Table

Route tables (1/5) Info

Last updated less than a minute ago | Actions | Create route table

Find route tables by attribute or tag

Name	Route table ID	Explicit subnet associ...	Edge associations	Main	VI
project-vpc-b00169348-rtb-private2-us...	rtb-07591ba2540826f42	–	–	No	vp
project-vpc-b00169348-rtb-public	rtb-0b182d5ae71b66e16	2 subnets	–	No	vp

rtb-0b182d5ae71b66e16 / project-vpc-b00169348-rtb-public

Explicit subnet associations (2)

Edit subnet associations

Find subnet association

Name	Subnet ID	IPv4 CIDR	IPv6 CIDR
project-public-subnet-1-b00169...	subnet-063dedcf3688eb4f4	10.0.1.0/24	–
project-public-subnet-2-b00169...	subnet-0c9fc635f4c6ce4b1	10.0.2.0/24	–

rtb-0b182d5ae71b66e16 / project-vpc-b00169348-rtb-public

Routes (2)

Both | Edit routes

Filter routes

Destination	Target	Status	Propagated	Route Origin
0.0.0.0/0	igw-013ed6e326da383f2	Active	No	Create Route
10.0.0.0/16	local	Active	No	Create Route Table

rtb-07591ba2540826f42 / project-vpc-b00169348-rtb-private2-us-east-1b

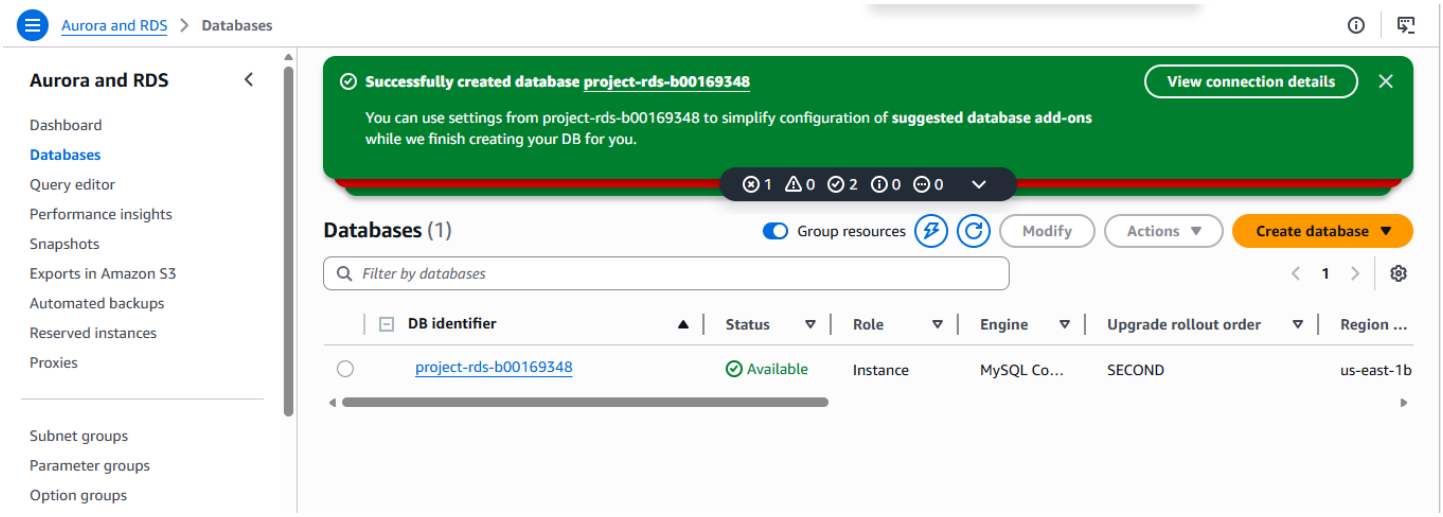
Explicit subnet associations (2)

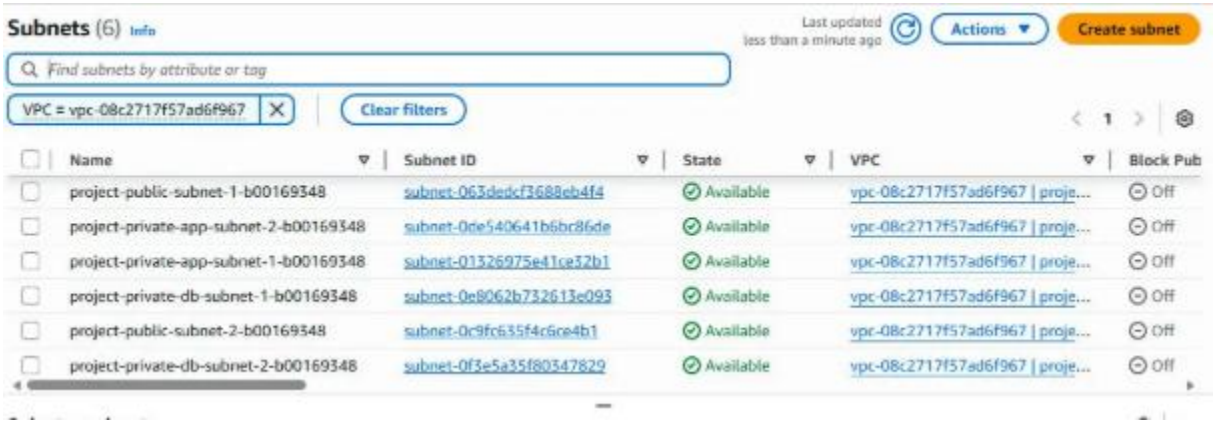
Edit subnet associations

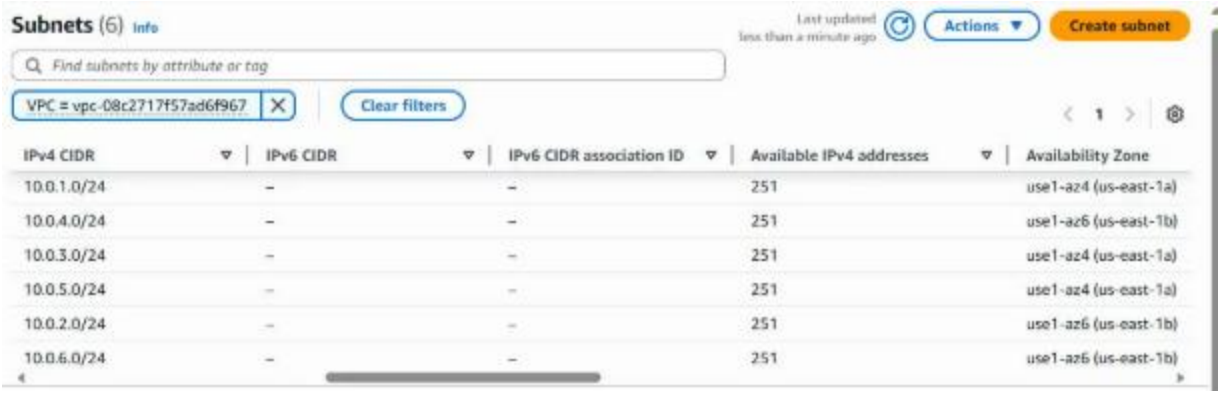
Find subnet association

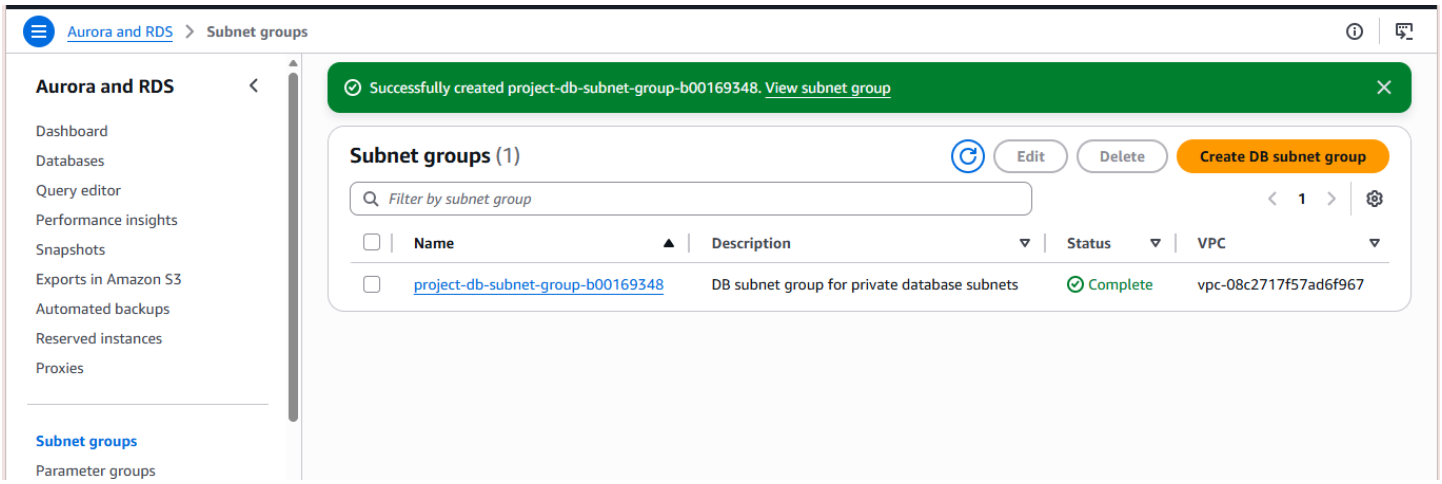
Name	Subnet ID	IPv4 CIDR	IPv6 CIDR
project-private-app-subnet-2-b0...	subnet-0de540641b6bc86de	10.0.4.0/24	–
project-private-db-subnet-2-b00...	subnet-0f3e5a35f80347829	10.0.6.0/24	–


rtb-07c830ec66bbf3fbe / project-vpc-b00169348-rtb-private1-us-east-1a
Explicit subnet associations (2)
Edit subnet associations
Find subnet association
Name
Subnet ID
IPv4 CIDR
IPv6 CIDR
project-private-app-subnet-1-b0...
subnet-01326975e41ce32b1
10.0.3.0/24
project-private-db-subnet-1-b00...
subnet-0e8062b732613e093
10.0.5.0/24


Aurora and RDS
Databases
Aurora and RDS
Dashboard
Databases
Query editor
Performance insights
Snapshots
Exports in Amazon S3
Automated backups
Reserved instances
Proxies
Subnet groups
Parameter groups
Option groups
Successfully created database project-rds-b00169348
View connection details
You can use settings from project-rds-b00169348 to simplify configuration of suggested database add-ons while we finish creating your DB for you.
Databases (1)
Group resources
Modify
Actions
Create database
Filter by databases
DB identifier
Status
Role
Engine
Upgrade rollout order
Region ...
project-rds-b00169348
Available
Instance
MySQL Co...
SECOND
us-east-1b


Subnets (6) Info
Last updated less than a minute ago
Actions
Create subnet
Find subnets by attribute or tag
VPC = vpc-08c2717f57ad6f967
Clear filters
Name
Subnet ID
State
VPC
Block Pub
project-public-subnet-1-b00169348
subnet-063dedcf3688eb4f4
Available
vpc-08c2717f57ad6f967 | proje...
Off
project-private-app-subnet-2-b00169348
subnet-0de540641b6bc86de
project-private-app-subnet-1-b00169348
subnet-01326975e41ce32b1
project-private-db-subnet-1-b00169348
subnet-0e8062b732613e093
project-public-subnet-2-b00169348
subnet-0c9fc635f4c6ce4b1
project-private-db-subnet-2-b00169348
subnet-0f3e5a35f80347829


Subnets (6) Info
Last updated less than a minute ago
Actions
Create subnet
Find subnets by attribute or tag
VPC = vpc-08c2717f57ad6f967
Clear filters
1
IPv4 CIDR
IPv6 CIDR
IPv6 CIDR association ID
Available IPv4 addresses
Availability Zone
10.0.1.0/24 – – 251 use1-az4 (us-east-1a)
10.0.4.0/24 – – 251 use1-az6 (us-east-1b)
10.0.3.0/24 – – 251 use1-az4 (us-east-1a)
10.0.5.0/24 – – 251 use1-az4 (us-east-1a)
10.0.2.0/24 – – 251 use1-az6 (us-east-1b)
10.0.6.0/24 – – 251 use1-az6 (us-east-1b)


Aurora and RDS > Subnet groups
Aurora and RDS
Dashboard
Databases
Query editor
Performance insights
Snapshots
Exports in Amazon S3
Automated backups
Reserved instances
Proxies
Subnet groups
Parameter groups
Successfully created project-db-subnet-group-b00169348. View subnet group
Subnet groups (1)
Edit
Delete
Create DB subnet group
Filter by subnet group
1
Name
Description
Status
VPC
project-db-subnet-group-b00169348
DB subnet group for private database subnets
Complete
vpc-08c2717f57ad6f967